# MINUTES OF MEETING (MOM)
# Team 8
# 3D Printing Cost Estimation Platform

## Meeting Details

| | |
|---|---|
| **Project:** | 3D Printing Cost Estimation Platform |
| **Week:** | Week 5 |
| **Date:** | Thursday, February 20, 2026 |
| **Time:** | 4:00 PM IST |
| **Location:** | CIE, Ground Floor |
| **Client:** | Anil Kumar Upadhyay, Five Fingers Innovative Solutions |
| **Client Email:** | aku1974india@gmail.com |
| **Client Phone:** | 9949497560 |

### Team Members Present

- Mehrish Khan
- Ronith Menneni

## Meeting Objective

This week's meeting focused on reinforcing development best practices, addressing Sprint 2 progress, and clarifying processes for Release 1 delivery. Client emphasized the importance of proper version control, issue tracking, continuous integration, and comprehensive documentation.

## 1 Sprint 2 Progress Review

### 1.1 Completed Work

- **[3DP-10] CURA Slicing Integration:** 80% complete
  - CuraEngine Docker container operational
  - Basic slicing endpoint functional
  - G-code parsing in progress
  - Remaining: Error handling and failure logs
- **[3DP-15] 3D Model Viewer:** 70% complete
  - Three.js scene rendering STL files
  - OrbitControls working smoothly
  - Grid helper and axis indicators added
  - Remaining: Model dimensions display, performance optimization

- **[3DP-20] Slicing Parameters Form:** 90% complete
  - UI form with all CURA parameters
  - Frontend validation implemented
  - API integration working
  - Remaining: Loading states and error feedback
- **[3DP-25] Cost Calculation Engine:** 60% complete
  - Formula implementation complete
  - Material cost calculation working
  - Remaining: Machine time integration, breakdown UI

### 1.2 Blockers & Issues

- **[BLOCKER] CuraEngine G-code parsing edge cases:**
  - Some STL files generate malformed G-code comments
  - Workaround: Skip lines that don't match expected format
  - Permanent fix: Update parser with robust regex patterns
- **[ISSUE] Three.js memory leak on multiple file loads:**
  - Browser crashes after 5-6 consecutive uploads
  - Root cause: Not disposing previous geometry/materials
  - Fix in progress: Call dispose() before loading new model
- **[ISSUE] PostgreSQL connection pool exhaustion:**
  - Errors after 10 concurrent requests
  - Solution: Increase pool size from 5 to 15 connections
  - Already implemented and tested

## 2 GitHub Workflow Best Practices (Client Emphasis)

### 2.1 Repository Structure Enforcement

- **main branch:** Production-ready code only
  - Protected: No direct pushes allowed
  - Merge source: release branches only
  - Tagged commits for all releases
- **dev branch:** Integration branch for active development
  - All feature branches created from dev
  - Contains baseline setup code (project skeleton, configs)
  - Must be stable enough for team to pull from
- **Feature branches:** Individual developer workspaces
  - Naming: `feature/3DP-XX-descriptive-name`
  - Short-lived: Merge within 2-3 days
  - Deleted after merge to keep repo clean

### 2.2 Initial Setup Workflow (For New Projects)

1. Create **main** branch (default)
2. Create **dev** branch from main
3. Add baseline code to dev branch:
   - Project directory structure
   - Docker Compose configuration
   - Requirements files (Python, Node.js)
   - CI/CD workflow files (.github/workflows/)
   - README with setup instructions
   - .gitignore, .env.example
4. Protect main branch (require PR + approvals)
5. All team members pull from dev to start work

### 2.3 Development Workflow (Daily Operations)

1. Pull latest changes from dev:

```
git checkout dev
git pull origin dev
```

2. Create feature branch for assigned Jira issue:

```
git checkout -b feature/3DP-XX-short-description
```

3. Make changes and commit frequently:

```
git add .
git commit -m "[3DP-XX] Descriptive commit message"
```

4. Push feature branch to remote:

```
git push origin feature/3DP-XX-short-description
```

5. Create Pull Request on GitHub:
   - Base branch: `dev`
   - Compare branch: `feature/3DP-XX-...`
   - Title: "[3DP-XX] Feature name"
   - Description: What changed, why, how to test
   - Link Jira issue in description
   - Request review from team lead or peer
6. Address review comments, update PR
7. Team lead approves PR after:

- CI checks pass (lint, format, tests)
- Code review approval
- No merge conflicts with dev

8. Merge to dev (use "Squash and merge" for clean history)
9. Delete feature branch after merge
10. Pull merged changes back to local dev:

```
git checkout dev
git pull origin dev
```

### 2.4 Avoiding Common Mistakes

- **DON'T** work directly on dev or main branches
- **DON'T** force push to shared branches (`git push -f`)
- **DON'T** commit secrets (.env files, API keys)
- **DON'T** leave stale branches (delete after merge)
- **DON'T** create PRs without testing locally first
- **DO** pull from dev before starting new work
- **DO** write clear commit messages
- **DO** rebase if dev has moved ahead (avoid merge commits)
- **DO** communicate blockers immediately in Jira

## 3 Jira Issue Tracking (Client Requirements)

### 3.1 Creating and Managing Issues

- **All work starts with a Jira issue**
  - No "off-Jira" work allowed
  - If not in Jira, it doesn't exist
- **Issue Details Must Include:**
  - Clear title (e.g., "Implement file upload validation")
  - Detailed description (what needs to be done)
  - Acceptance criteria (definition of done)
  - Story points (estimated effort)
  - Labels (backend, frontend, bug, urgent, etc.)
  - Assignee (who will work on it)
  - Sprint assignment
  - Linked epic
- **Update Issue Status Promptly:**
  - To Do → In Progress (when you start coding)
  - In Progress → Code Review (when PR created)

  - Code Review → Testing (after merge to dev)
  - Testing → Done (after QA validation passes)

- **Document Blockers:**
  - Set status to "Blocked" if can't proceed
  - Add comment explaining blocker
  - Link blocking issue if applicable
  - Notify team lead immediately
  - Update in daily standup

### 3.2 Issue Prioritization

- **P0 (Critical):** Blocks release, fix immediately (database down, critical bug)
- **P1 (High):** Important feature, complete this sprint (CURA integration)
- **P2 (Medium):** Nice to have, next sprint if not done (admin panel)
- **P3 (Low):** Future enhancement (email templates customization)

### 3.3 Writing Effective MOMs in Jira

- Create "Meeting Notes" issue type in Jira
- Template:
  - Meeting date, time, attendees
  - Agenda items
  - Discussion summary
  - Decisions made
  - Action items (link to issues)
  - Next meeting date
- Tag with "MOM" label for easy filtering
- Link related issues raised during meeting

### 3.4 Tracking Individual Contributions

- Use Jira "Sprint Report" to view completed work
- GitHub Insights shows commit/PR statistics
- Weekly review in standup:
  - How many issues completed?
  - How many PRs merged?
  - Code review participation?
  - Test contributions?
- Goal: Each member completes 2-3 issues per sprint

# 4 CI/CD Pipeline Requirements

## 4.1 Continuous Integration Goals

- **Fast feedback:** CI runs complete in ¡ 5 minutes
- **Catch issues early:** Every PR triggers CI before review
- **Enforce standards:** Code must pass all checks to merge
- **Build verification:** Ensure code compiles/builds successfully

## 4.2 CI Checks (Must All Pass)

1. **Linting:** Code style compliance
   - Python: pylint, flake8
   - JavaScript/TypeScript: ESLint
   - Fail if linting score ¡ 8/10
2. **Formatting:** Consistent code format
   - Python: black –check
   - JavaScript: prettier –check
   - Fail if any file needs formatting
3. **Type Checking:** Catch type errors
   - Python: mypy (strict mode)
   - TypeScript: tsc –noEmit
4. **Unit Tests:** Verify functionality
   - Python: pytest (all tests must pass)
   - JavaScript: vitest/jest
   - Minimum 80% code coverage
5. **Integration Tests:** API endpoint tests
   - Test with real database (separate test DB)
   - Verify request/response schemas
6. **Docker Build Check:** Ensure images build
   - Run docker-compose build
   - Fail if any service fails to build
   - Catch dependency issues early

## 4.3 Package Management Best Practices

- **Lock dependency versions:**
  - Python: `requirements.txt` with exact versions (==)
  - Node.js: Commit `package-lock.json`
- **Regular dependency updates:**
  - Check for security vulnerabilities weekly
  - Update patch versions promptly

  - Test thoroughly after major version updates

- **Build reproducibility:**
  - Docker images use specific base image tags (e.g., `python:3.11.8`)
  - Never use `latest` tag in production

### 4.4 Build and Run Before Submitting PR

- **Developer responsibility:**
  1. Run tests locally: `pytest tests/`
  2. Run linting: `black . && pylint app/`
  3. Build Docker images: `docker-compose build`
  4. Start full stack: `docker-compose up`
  5. Manually test changed functionality
  6. Only then create PR
- **CI is NOT a substitute for local testing**
- **If CI fails:** Fix immediately, don't wait for reviewer

## 5 Release Management (Detailed Process)

### 5.1 Release 1 Demo Deadline (Reconfirmed)

- **Date:** Friday, February 28, 2026 at 4:00 PM IST
- **Deliverables:**
  - Working end-to-end demo (upload → slice → view → estimate)
  - Docker Compose brings up entire stack
  - All P0/P1 bugs fixed
  - Documentation complete (README, API docs, diagrams)
  - Release notes prepared
  - Git tag created: `v1.0.0`

### 5.2 Release Branch Workflow

1. **Code Freeze:** Wednesday, Feb 26 at 6:00 PM
   - No new features after this point
   - Only bug fixes allowed in release branch
2. **Create Release Branch:**

```
git checkout dev
git pull origin dev
git checkout -b release/v1.0.0
git push origin release/v1.0.0
```

3. **QA Testing (Feb 26-27):**
   - Run complete test suite (unit + integration + E2E)
   - Manual testing: All features in acceptance criteria

- Cross-browser testing (Chrome, Firefox, Safari)
- Performance testing (load 10MB STL, measure time)
- Document all bugs found

4. **Bug Fixing (Feb 27-28):**

   - Fix P0/P1 bugs directly in release branch
   - Quick PR review (¡ 2 hours turnaround)
   - Retest after each fix

5. **Merge to Main (Feb 28 morning):**

```
git checkout main
git merge --no-ff release/v1.0.0
git push origin main
```

6. **Create Release Tag:**

```
git tag -a v1.0.0 -m "Release 1.0.0 - MVP with upload, slice, view, estimate"
git push origin v1.0.0
```

7. **Merge Back to Dev:**

```
git checkout dev
git merge main
git push origin dev
```

8. **Create GitHub Release:**

   - Tag: v1.0.0
   - Title: "Release 1.0.0 - MVP Features"
   - Description: Paste release notes (see template below)
   - Attach build artifacts (optional): Docker images, compiled bundles

### 5.3 Release Notes Structure

```
# Release v1.0.0 - February 28, 2026

## Overview
First production release (MVP) of 3D Printing Cost Estimation Platform
for Five Fingers Innovative Solutions.

## New Features
- [3DP-1] File upload API (STL/STEP, max 50MB, validation)
- [3DP-10] CURA slicing engine integration (G-code generation)
- [3DP-15] Interactive 3D model viewer (Three.js, rotate/zoom/pan)
- [3DP-20] Slicing parameter configuration (material, layer height, infill)
- [3DP-25] Automated cost estimation (material + machine time)
- [3DP-30] Simple authentication (username/password)
- [3DP-35] Admin material configuration panel
```

```
## Bug Fixes
- [3DP-BUG-1] Fixed PostgreSQL permission errors in Docker setup
- [3DP-BUG-2] Resolved Three.js memory leak on multiple uploads
- [3DP-BUG-3] Fixed G-code parser handling malformed comments
- [3DP-BUG-4] Corrected cost calculation rounding errors

## Known Issues
- Large STL files (>50MB) may cause browser slowdown during render
- G-code layer preview limited to first 1000 layers for performance
- Email notifications not yet implemented (planned for R2)

## Testing Results
**Unit Tests:**
- Backend: 52/52 passed (85% coverage)
- Frontend: 28/28 passed (72% coverage)

**Integration Tests:**
- API endpoints: 15/15 passed
- Database operations: 8/8 passed

**E2E Tests:**
- Upload workflow: PASSED
- Slice workflow: PASSED
- Cost estimation: PASSED

**Manual QA:**
- Cross-browser (Chrome, Firefox, Safari): PASSED
- Upload 5MB STL: < 2 seconds PASSED
- Slice to G-code: 45 seconds PASSED
- 3D viewer performance: 60fps PASSED

## Deployment Instructions
1. Clone repository: git clone <repo-url>
2. Install Docker 20.10+ and Docker Compose 2.0+
3. Copy .env.example to .env and configure
4. Run: docker-compose up -d
5. Access:
   - Frontend: http://localhost:3000
   - Backend API: http://localhost:8000
   - Swagger Docs: http://localhost:8000/docs
6. Create admin user: docker exec backend python create_admin.py

## System Requirements
- Docker 20.10+
- Docker Compose 2.0+
- 4GB RAM minimum (8GB recommended)
- Modern web browser (Chrome 90+, Firefox 88+, Safari 14+)

## Breaking Changes
None (initial release)
```

```
## Contributors
Team 8 (IIIT Hyderabad):
- Abhiroop Pullepu (Backend Lead)
- Mehrish Khan (Backend Developer)
- Ruschita Guddeti (Project Manager)
- Sai Hasini (Frontend Developer)
- Ronith Menneni (Frontend Developer)

Client: Anil Kumar Upadhyay (Five Fingers Innovative Solutions)

## Next Release (v1.1.0 - Planned for March 15, 2026)
- Email notifications (Mailtrap integration)
- G-code preview with layer-by-layer animation
- Advanced model analysis (thin walls, overhangs detection)
- Export cost estimate as PDF
- Job history and search
```

### 5.4 Hotfix Process (For Production Bugs)

- **When:** Critical bug found in production (main branch)
- **Severity:** P0 only (system down, data loss, security issue)
- **Timeline:** Fix within 24 hours
- **Process:**
  1. Create hotfix branch from main: `git checkout -b hotfix/v1.0.1 main`
  2. Fix bug with minimal changes
  3. Test thoroughly (automated + manual)
  4. Create PR to main (expedited review)
  5. Merge to main after approval
  6. Tag as v1.0.1: `git tag -a v1.0.1 -m "Hotfix: Critical bug description"`
  7. Push tag: `git push origin v1.0.1`
  8. Merge hotfix back to dev: `git checkout dev && git merge main`
  9. Update release notes (mention hotfix)
  10. Notify client and team

### 5.5 Release Numbering Convention

- **Format:** vMAJOR.MINOR.PATCH (Semantic Versioning)
- **MAJOR (v2.0.0):** Breaking API changes, major architecture overhaul
- **MINOR (v1.1.0):** New features, backward compatible
- **PATCH (v1.0.1):** Bug fixes, security patches, no new features
- **Examples:**
  - v1.0.0 → v1.0.1 (hotfix for bug)
  - v1.0.1 → v1.1.0 (added email notifications)
  - v1.1.0 → v2.0.0 (changed API from REST to GraphQL)

# 6 Testing Strategy (Comprehensive)

## 6.1 Test Case Management

- **Document test cases in Jira:**
  - Create issue type “Test Case”
  - Link to user story being tested
  - Include: Test steps, expected result, actual result
  - Status: Not Tested, Passed, Failed, Blocked
- **Test Execution:**
  - Before release, execute all test cases
  - Update Jira with pass/fail status
  - Create bug issues for failures
  - Retest after bug fixes

## 6.2 Automated Testing Requirements

- **Unit Tests (80%+ coverage):**
  - Test individual functions in isolation
  - Mock external dependencies (database, API calls)
  - Fast execution (¡ 5 seconds total)
  - Run automatically in CI on every commit
- **Integration Tests:**
  - Test API endpoints with real test database
  - Verify database transactions
  - Check request/response formats
  - Run in CI on PRs to dev/main
- **E2E Tests (Playwright/Cypress):**
  - Test complete user workflows
  - Upload file → configure slicing → view model → get estimate
  - Run before releases only (time-consuming)
  - Catch integration issues between frontend/backend

## 6.3 Manual Testing Checklist

- Upload STL file (valid, invalid, oversized)
- View 3D model (rotate, zoom, pan)
- Configure slicing parameters (all dropdowns/inputs)
- Trigger slicing (success and failure cases)
- View G-code preview
- Check cost estimate calculation
- Admin: Add/edit/delete materials
- Admin: Update machine rates

- Test on Chrome, Firefox, Safari
- Test on different screen sizes (desktop, tablet)

### 6.4 Bug Severity & Priority

- **P0 (Critical):** System down, data loss, security vulnerability
  - Fix immediately (within 24 hours)
  - Blocks release
- **P1 (High):** Major feature broken, significant impact
  - Fix before release
  - May delay release if not resolved
- **P2 (Medium):** Minor feature issue, workaround exists
  - Fix in next sprint
  - Doesn't block release
- **P3 (Low):** Cosmetic issue, typo, minor enhancement
  - Fix when convenient
  - Backlog item

## 7 Documentation Requirements

### 7.1 Control Flow Diagram

- **Tool:** Mermaid.js (stored in `/docs/diagrams/control-flow.md`)
- **Content:**
  - User uploads file → Validation → Storage → Database record
  - User configures parameters → Slicing request → CURA execution
  - G-code generation → Parsing → Material/time calculation
  - Cost calculation → Breakdown display
- **Status:** In progress (Ruschita)
- **Due:** February 25, 2026

### 7.2 Sequence Diagrams (4 Required)

1. **File Upload Sequence:**
   - User → Frontend → Backend API → File System → Database
   - Include: Validation steps, error handling
2. **Slicing Sequence:**
   - User → Frontend → Backend → CURA Docker → G-code Parser → Database
   - Include: Timeout handling, failure logging
3. **Cost Estimation Sequence:**
   - User request → Backend → Database (fetch slicing results + material rates)

- Cost calculation → Return breakdown

4. **Authentication Sequence:**
   - User login → Backend → Database (verify credentials)
   - Generate session token → Return to frontend
   - Frontend includes token in subsequent requests

### 7.3 Use Case Diagram

- **Already completed:** Included in SRS document
- **Actors:** Customer, Administrator, System (CURA Engine)
- **Update if needed:** Add new use cases discovered during development

### 7.4 README Documentation

- **/README.md (Root):**
  - Project overview
  - Features list
  - Tech stack
  - Quick start guide (docker-compose up)
  - Links to detailed docs
- **/backend/README.md:**
  - API documentation (link to Swagger)
  - Database schema
  - Environment variables
  - Running tests
- **/frontend/README.md:**
  - Component structure
  - State management
  - Building for production
  - Environment configuration

## 8 Issue Escalation Process

### 8.1 When to Escalate

- **Blocker persists for ¿ 24 hours:** Cannot proceed with assigned task
- **Technical uncertainty:** Don't know how to implement feature
- **Resource conflicts:** Need input from another team member who's unavailable
- **Scope creep:** Client requests features not in original plan
- **Timeline concerns:** Won't meet sprint deadline

### 8.2 Escalation Steps

1. **Mark issue as "Blocked" in Jira**
2. **Add detailed comment:** What's blocking you, what you've tried
3. **Notify team lead:** Via Slack/email immediately
4. **Discuss in daily standup:** Team may have solutions
5. **Manager escalation:** If blocker affects multiple team members or timeline
6. **Client escalation:** If blocker requires client decision or clarification

### 8.3 Documenting Issues

- **Create "Issue Report" in Jira:**
  - Issue type: Bug or Impediment
  - Priority: Based on impact
  - Description: Detailed explanation with screenshots/logs
  - Steps to reproduce (if bug)
  - Environment details (OS, browser, Docker version)
  - Attempted solutions
- **Link to affected user stories**
- **Update weekly in status report to manager**
- **Track resolution time** (for retrospective analysis)

## 9 Individual Contribution Visibility

### 9.1 Metrics for Tracking

- **GitHub Contributions:**
  - Commits: Quantity and quality (meaningful changes)
  - Pull Requests: Number merged, average review time
  - Code Reviews: How many PRs reviewed, feedback quality
  - Lines of Code: Context matters (100 lines of clean code ¿ 500 lines messy)
- **Jira Activity:**
  - Issues completed per sprint
  - Story points delivered
  - Time logged accurately
  - Comments/updates on issues
- **Testing Contributions:**
  - Unit tests written
  - Test coverage increased
  - Bugs found and reported
- **Documentation:**
  - README updates

  - Code comments (where needed)
  - API documentation
  - Diagrams created

### 9.2 Weekly Review Process

- **Team Lead Responsibilities:**

  1. Review GitHub Insights dashboard
  2. Check Jira Sprint Report
  3. Identify low contributors early
  4. Discuss with individual: Are there blockers? Need help?
  5. Reassign work if needed

- **Red Flags:**

  - 0-1 commits in a week (without valid reason)
  - No PR activity for 5+ days
  - Issues stuck in "In Progress" for ¿ 5 days
  - Not participating in code reviews

### 9.3 Ensuring Fair Distribution

- Assign roughly equal story points per person
- Mix of challenging and routine tasks
- Frontend/backend members get domain-appropriate work
- Everyone gets opportunity to work on interesting features
- Pair programming for complex features

## 10 Communication & Collaboration

### 10.1 Daily Standups (Reinforced)

- **Time:** 10:00 AM IST, 15 minutes maximum
- **Format:** Each member answers:

  1. What I completed yesterday
  2. What I'll work on today
  3. Any blockers or help needed

- **Update Jira before standup:** Move issues to correct status
- **No problem-solving in standup:** Take detailed discussions offline

### 10.2 Code Review Etiquette

- **Reviewer Responsibilities:**

  - Respond within 24 hours (48 hours max)
  - Provide constructive feedback
  - Approve if no blocking issues
  - Request changes if significant problems

  - Test locally if unsure about functionality

- **Author Responsibilities:**
  - Address feedback promptly
  - Explain reasoning if you disagree politely
  - Don't take criticism personally
  - Thank reviewers for their time
  - Update PR description if scope changes

### 10.3 Slack/Communication Channels

- **#general:** Announcements, general discussion
- **#backend:** Backend-specific technical questions
- **#frontend:** Frontend implementation discussions
- **#ci-cd:** Build failures, CI issues
- **#blockers:** Immediate attention needed
- **Direct messages:** For 1-on-1 quick questions

## 11 Sprint 2 Goals (Week 5 Continued)

### 11.1 This Week (Feb 20-26)

- Complete all Sprint 2 stories (100%)
- Achieve 80%+ test coverage on backend
- Fix all known P1/P2 bugs
- Complete control flow and sequence diagrams
- Conduct internal QA testing
- Prepare demo environment

## 12 Next Meeting

- **Date:** Friday, February 28, 2026 (Release 1 Demo)
- **Time:** 4:00 PM IST
- **Location:** CIE Ground Floor (with projector)
- **Agenda:**
  - Demonstrate complete end-to-end workflow
  - Show all completed features
  - Review release notes with client
  - Discuss Sprint 3 priorities
  - Get client approval for Release 1

**Minutes prepared by:** Team 8
**Date:** February 20, 2026